Introducción a las funciones con Meta AI

Tipología de funciones en Python

Actúa como tutor de Python para Ciencia de Datos. Explícame detalladamente en qué se diferencian los distintos tipos de funciones en Python (por ejemplo: built-in, definidas por el usuario con def, lambda/anónimas, métodos (instancia/clase/estático), generadoras con yield y funciones async). Para cada tipo: (1) definición breve, (2) un ejemplo corto en Python, y (3) un caso real de cuándo usarla

1. Funciones Built-in (Integradas)

- **Definición breve:** Son funciones que ya vienen predefinidas en Python y están siempre disponibles para usar sin necesidad de importarlas. Son herramientas esenciales para tareas comunes.
- **Ejemplo corto en Python:**

```
# Usando la función built-in 'len()'
lista_datos = [10, 20, 30, 40, 50]
longitud = len(lista_datos)
print(f"La longitud de la lista es: {longitud}")
# Usando la función built-in 'sum()'
total = sum(lista_datos)
print(f"La
```

8 lines (7 loc) · 288 B

- **Caso real en Ciencia de Datos:** Son el pan de cada día. Las usarías constantemente para operaciones básicas como:
  - `len()`: Para obtener el número de filas en un DataFrame de Pandas (`len(df)`) o el número de elementos en una lista.
  - `print()`: Para inspeccionar tus datos o depurar tu código.
  - `max()`, `min()`, `sum()`: Para calcular rápidamente estadísticas descriptivas sobre una serie de datos.
  - `type()`: Para verificar el tipo de dato de una variable o columna.
  - `int()`, `float()`, `str()`: Para convertir tipos de datos, lo cual es crucial en la limpieza de datos.

2. Funciones Definidas por el Usuario (`def`)

- **Definición breve:** Son bloques de código que escribes tú mismo para realizar una tarea específica y reutilizable. Se definen con la palabra clave `def`.
- **Ejemplo corto en Python:**

Arquitectura y diseño de funciones:

Actúa como ingeniero/a de software. ¿Bajo qué criterios técnicos es preferible que una función definida por el usuario reciba parámetros o no? Incluye aspectos como acoplamiento, pureza, dependencias, configuración, testabilidad y reutilización. Además, ¿en qué escenarios es mejor retornar un valor frente a solo ejecutar una acción con efectos secundarios? Da ejemplos concretos en Python

escenarios es mejor retornar un valor frente a solo ejecutar una acción con efectos secundarios? Da ejemplos concretos en Python

Hoy

¡Claro! Como ingeniera de software, te diría que la decisión de si una función debe recibir parámetros o no, y si debe retornar un valor o generar efectos secundarios, es fundamental para el diseño de software robusto, mantenible y escalable. Vamos a desglosarlo con criterios técnicos claros.

Criterios para la Recepción de Parámetros en Funciones Definidas por el Usuario

La decisión de si una función debe recibir parámetros se reduce a qué datos necesita operar y de dónde los obtiene. Los principales criterios técnicos son:

1. **Pureza de la Función y Determinismo:**
   - **Con Parámetros:** Una función es considerada "pura" si, dados los mismos parámetros de entrada, siempre produce la misma salida y no tiene efectos secundarios (no modifica variables externas ni interactúa con el mundo exterior). Esto hace que la función sea **determinista** y predecible.
   - **Sin Parámetros (o que usa datos globales/externos):** Si una función accede a variables globales, configura directamente recursos externos (como un archivo o una conexión a base de datos), o depende del estado de algún objeto fuera de su ámbito, su comportamiento puede variar incluso con la misma llamada. Esto reduce la pureza y el determinismo.
   - **Criterio:** Prefiere parámetros para fomentar la pureza. Si una función necesita datos para operar, es mejor que los reciba explícitamente como parámetros.
2. **Acoplamiento (Coupling):**
   - **Con Parámetros:** Reduce el acoplamiento. La función declara explícitamente sus dependencias a través de sus parámetros. No necesita conocer la estructura del programa ni dónde se almacenan esos datos; solo necesita los datos en sí. Esto facilita que la función sea usada en diferentes contextos.
   - **Sin Parámetros (o que usa datos globales/externos):** Aumenta el acoplamiento. La función está

UDF vs. funciones lambda

Compara funciones definidas por el usuario (UDF con def) vs funciones lambda en Python. ¿En qué situaciones específicas es preferible usar def en lugar de lambda? Explica el impacto en legibilidad, depuración, documentación, pruebas y reutilización. Incluye un ejemplo con lambda y su refactor a def (y explica por qué mejora)

¡Excelente pregunta! Es un punto crucial en el diseño y la ingeniería de software en Python. Tanto las funciones `def` como las `lambda` son herramientas poderosas para definir lógica, pero tienen propósitos y casos de uso óptimos muy distintos.

Vamos a compararlas y ver cuándo es mejor usar `def` en lugar de `lambda`.

Comparación: Funciones `def` vs. Funciones `lambda`

Característica	`def` (Función Definida por el Usuario)	`lambda` (Función Anónima)

Sintaxis	`def nombre_funcion(parametros):`
	`# Bloque de código`
	`return valor`
		`lambda parametros: expresion`

Nombre	Tiene un nombre explícito.	No tiene nombre (es anónima), aunque se le puede asignar a una variable.
Cuerpo de la Función	Puede contener múltiples expresiones, sentencias, bucles, manejo de errores, etc. (cuerpo complejo).	Solo puede contener una única expresión. Implica un `return` implícito del resultado de esa expresión. No puede tener sentencias complejas (asignaciones, `if/else` explícitos, `try/except`, `for`).
Docstrings	Soporta docstrings para documentar su propósito, parámetros y retorno.	No soporta docstrings de forma directa y limpia.
Depuración	Fácil de depurar: tiene un nombre, se puede establecer un breakpoint, los tracebacks son claros.	Más difícil de depurar: aparecen como `<lambda>` en los tracebacks, lo que dificulta identificar dónde ocurrió el error si tienes muchas.
Reutilización	Diseñada para ser reutilizada en múltiples lugares del código.	Generalmente se usa para un solo propósito o en un contexto específico (callbacks, `key` para `sorted`, `filter`, `map`).
Recursión	Puede ser recursiva.	Puede ser recursiva, pero de forma más limitada y menos legible (requiere asignarla a una variable).
Tipo de Objeto	Es un objeto de función normal.	Es un objeto de función, pero su propósito es ser concisa para casos específicos.